5.GPIO

> 学习目标：搞懂GPIO到底是什么、有几种工作模式、怎么用CubeMX配置、掌握点灯代码，尝试操作寄存器

读完本章之后，可以试着回答文末的思考题，用来检验自己有没有真正理解内容。

一、GPIO到底是什么?

GPIO 的全称是 General Purpose Input Output，翻译过来就是**通用输入输出引脚**，我们日常直接简称IO口，是单片机和外部硬件打交道最基础的通道。单片机可以通过这根触角向外输出高低电平，也可以读取外部送来的电平信号。

举两个生活里最容易理解的例子：

1. **输出场景**：我们最经典的“点灯实验”。把LED灯的一端接到GPIO引脚，单片机控制这个引脚输出高/低电平，就能控制LED亮或者熄灭。这就是GPIO作为输出口在工作。
2. **输入场景**：按键检测。按键一端接GPIO引脚，当我们按下按键时，外部电平发生变化，单片机能通过GPIO读取到这个变化，识别出“按键被按下”这个动作，这就是GPIO作为输入口在工作。

STM32的每一个GPIO引脚都不是只能固定做输入或者固定做输出，**同一个引脚可以通过配置寄存器，随时切换成输入、输出或者其他复用功能**，这也是“通用”两个字的含义。

STM32的引脚是分组管理的，我们常见的命名是 `PA0` 、 `PB5` 、 `PC13` 这种格式：

- 字母A/B/C/D…代表GPIO端口组，也就是GPIOA、GPIOB、GPIOC等；
- 后面的数字代表这个端口里面的引脚编号，从0开始排序。

比如PB5，就是GPIOB端口下的第5号引脚，很多开发板上面自带的板载LED，默认就连接在PB5这个引脚上，也是我们入门实验最常用的引脚。

二、GPIO的使用

GPIO的6种工作模式

STM32的GPIO一共支持6种工作模式，每一个模式本质上，都是芯片内部给这个引脚切换了一套不同的内部电路结构，从而实现不一样的功能。

1. **推挽输出（GPIO_Output_PP）**

推挽输出适合直接驱动LED、继电器这类负载。

简单理解：引脚输出高电平时，内部直接接到电源；输出低电平时，内部直接接到地。两种状态驱动能力都很强，可以直接输出稳定的高低电平。我们做点灯实验，默认就使用推挽输出模式。

2. **开漏输出（GPIO_Output_OD）**

开漏输出只能主动拉低电平，想要输出高电平，必须外部额外接上拉电阻。

单独使用的场景不多，它最大的用处是**多设备线与通信**，I2C通信底层就使用开漏模式。

3. **浮空输入（GPIO_Input_Floating）**

引脚内部不接上拉、也不接下拉电阻。如果外部没有任何设备接入，引脚的电平是不确定的，会随机漂移，数值不稳定。

适合外部电路已经自带稳定上拉/下拉的信号采集，比如部分传感器信号读取。按键检测一般不推荐浮空输入，很容易出现误触发。

4. **上拉输入（GPIO_Input_PU）按键首选**

引脚内部自带上拉电阻，默认情况下引脚电平是高电平；当外部把引脚接到地的时候，才会读到低电平。

按键实验最推荐这个模式：没有按下按键时，引脚默认高；按下按键，引脚接地变成低，单片机很稳定就能识别按键动作，不容易受外界干扰。

5. **下拉输入（GPIO_Input_PD）**

和上拉输入刚好相反：引脚内部自带下拉电阻，默认电平是低电平；外部给高电平信号时，单片机会读到高。适合默认低、外部触发给高的检测场景。

6. **复用功能（GPIO_AF）**

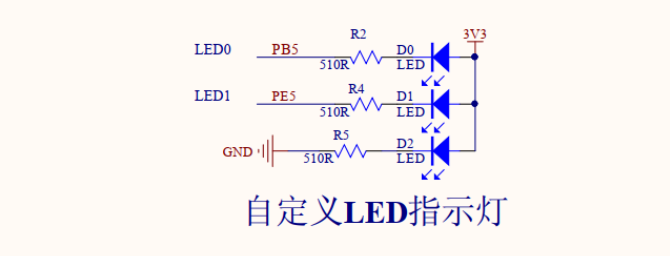
当引脚不做普通输入输出，交给片上其他外设使用时，就配置成复用模式。

比如串口、SPI、I2C、PWM这些外设，都需要占用GPIO引脚，此时这个引脚就不再由GPIO寄存器直接控制，而是交给对应的外设接管。后续学习通信、电机调速的时候会用到。

> 补充：GPIO输出还可以配置速度等级（低速、中速、高速），这个参数控制引脚电平翻转的最快速度。入门点灯实验直接默认低速就完全够用，不需要额外修改。速度越高，芯片的噪声和功耗也会变大。

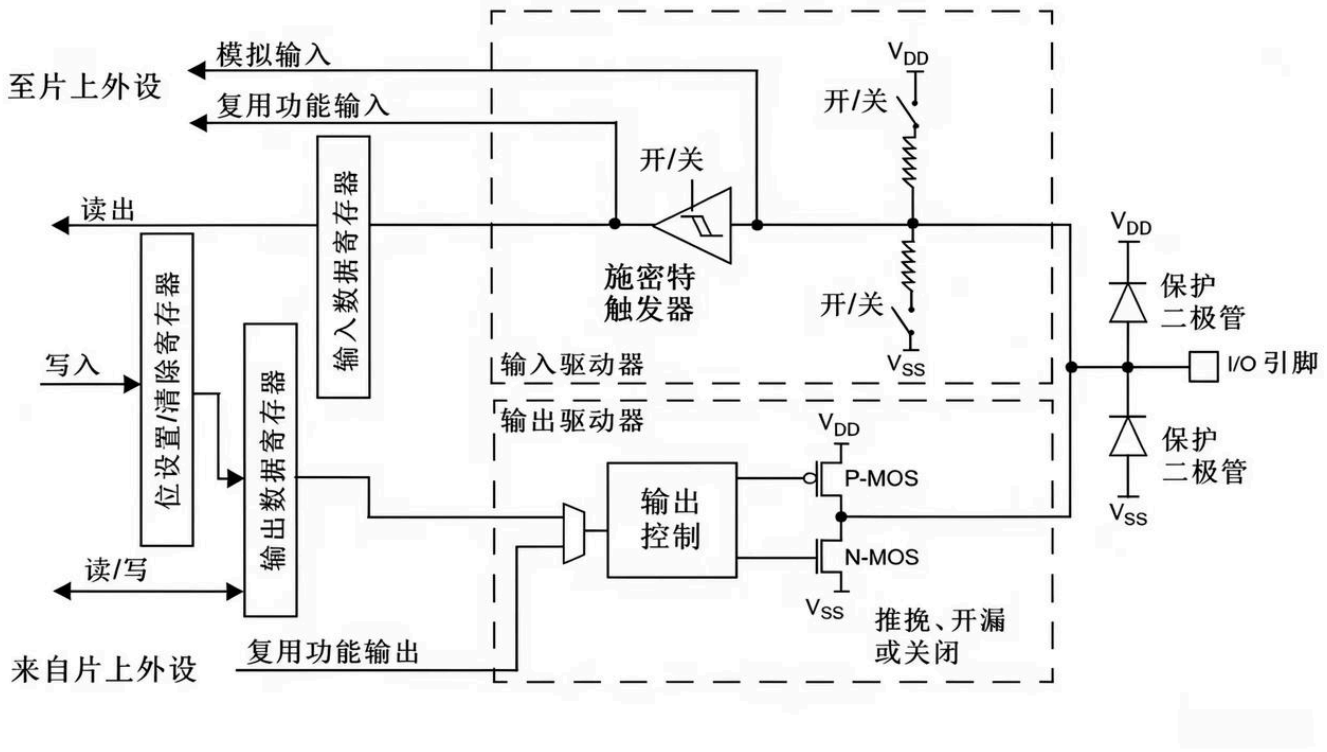
LED与GPIO

点亮LED的原理十分简单，只要开发板上面提供了用于用户使用的LED，从图上可以看出LED一端已经连接到高电平，此时只需要找到这颗LED连接在芯片的哪个引脚上，使用程序拉低这个引脚，LED两段有高低电压差，即可点亮，芯片上的可控制引脚就是GPIO。LED只是众多可以被GPIO操控的设备之一。



开发板的LED硬件连接原理图

图上蓝色三角形为发光二极管(LED)，折线为电阻，GND为接地（低电平），VCC为高电平，通常为3V3或5V。

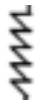


这是 STM32 单个 IO 口的内部电路，依靠寄存器配置，切换不同工作模式。

1. **外部入口**：两个保护二极管，防止外部过高 / 过低电压损坏芯片；还有可配置的内部上拉 / 下拉电阻。
2. **输入通路**引脚信号经施密特触发器整形后存入**输入数据寄存器**，CPU 读该寄存器就能获取引脚电平；也可以直接供给外设（复用输入）或 ADC（模拟输入，此时关闭数字整形电路）。
3. **输出通路**由**输出数据寄存器**或者片上外设信号控制一对 P/N MOS 管：

- 推挽输出：上下 MOS 配合，既能输出高、也能输出低，驱动能力强（LED 常用）
- 开漏输出：仅下 MOS 可控，只能主动拉低，高电平依靠外部上拉（I2C 总线使用）

4. **寄存器**：输出寄存器存要发出的电平，输入寄存器存读到的电平，置位 / 清零寄存器可以快速修改引脚状态。

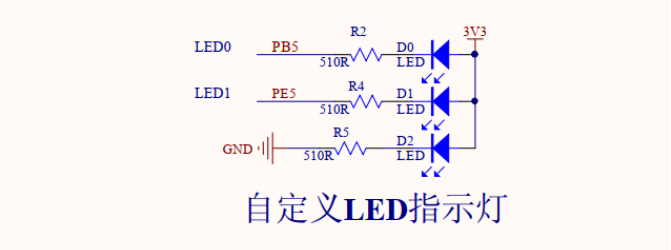
名词	解释
VDD	当作正极理解就好
VSS	当作负极就好
P-MOS	当作电子开关，受侧面信号控制上下两接口的通断，高电平导通
N-MOS	当作电子开关，与P-MOS的信号极性相反，低电平导通
保护二极管	保护不受高压烧毁
施密特触发器	电压输入超过一定阈值输出高电平，低于一定阈值输出低电平
	连接VDD的电阻称为上拉电阻，上拉电阻上面的开关闭合之后，能够让引脚浮空时候电平与VDD保持一致即高电平（没有电流，电阻压降为0），在引脚连接低电平的时候有从VDD流向低电平的电流。 同理，连接VSS的电阻称为下拉电阻，开关闭合之后能够让引脚浮空时保持低电平（下拉电阻压降为0）。引脚连接外部的电平之后，如果是高电平，产生从高电平流向VSS的电流，有电流有电阻，产生压降，让引脚能够被拉高。

GPIO点灯如上所说，应该将GPIO引脚拉低，了解到GPIO的各种模式之后我们发现推挽模式和开漏模式都可以将GPIO拉低，但是开漏模式如果不外接高电平我们关不掉LED，因此我们选择推挽。

三、用CubeMX配置GPIO点亮LED

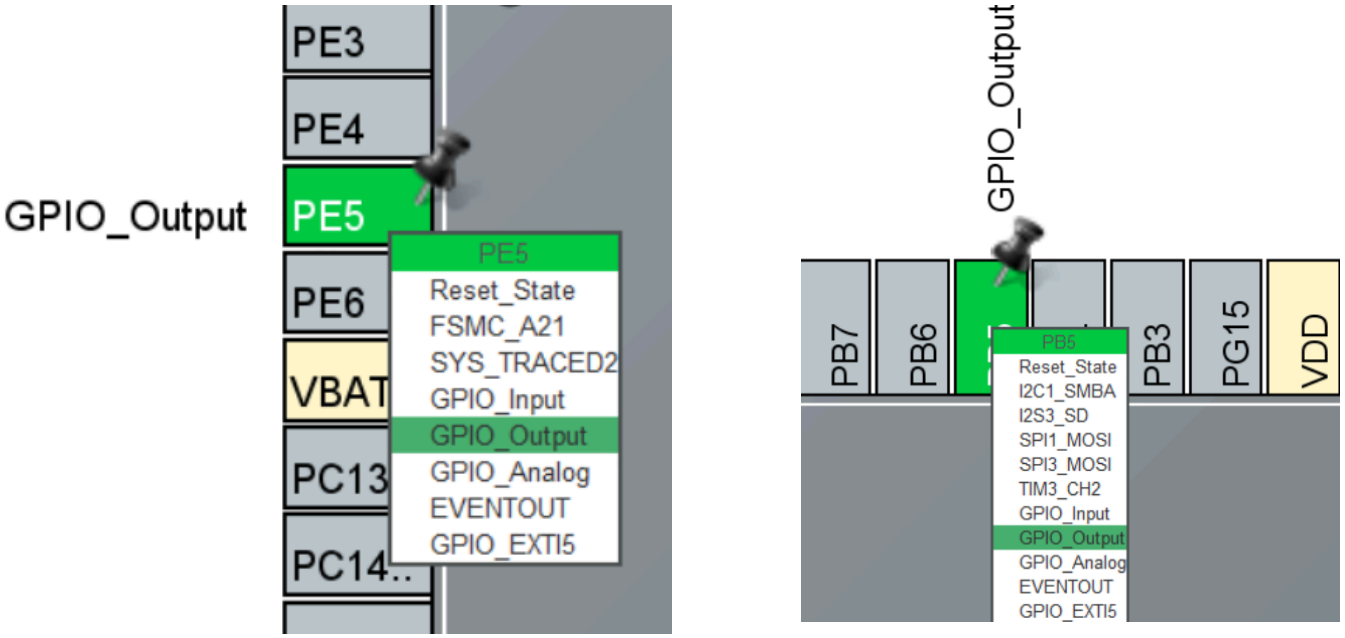
CubeMX配置

首先，参考通识的教程配置SYS和时钟，这里不再赘述

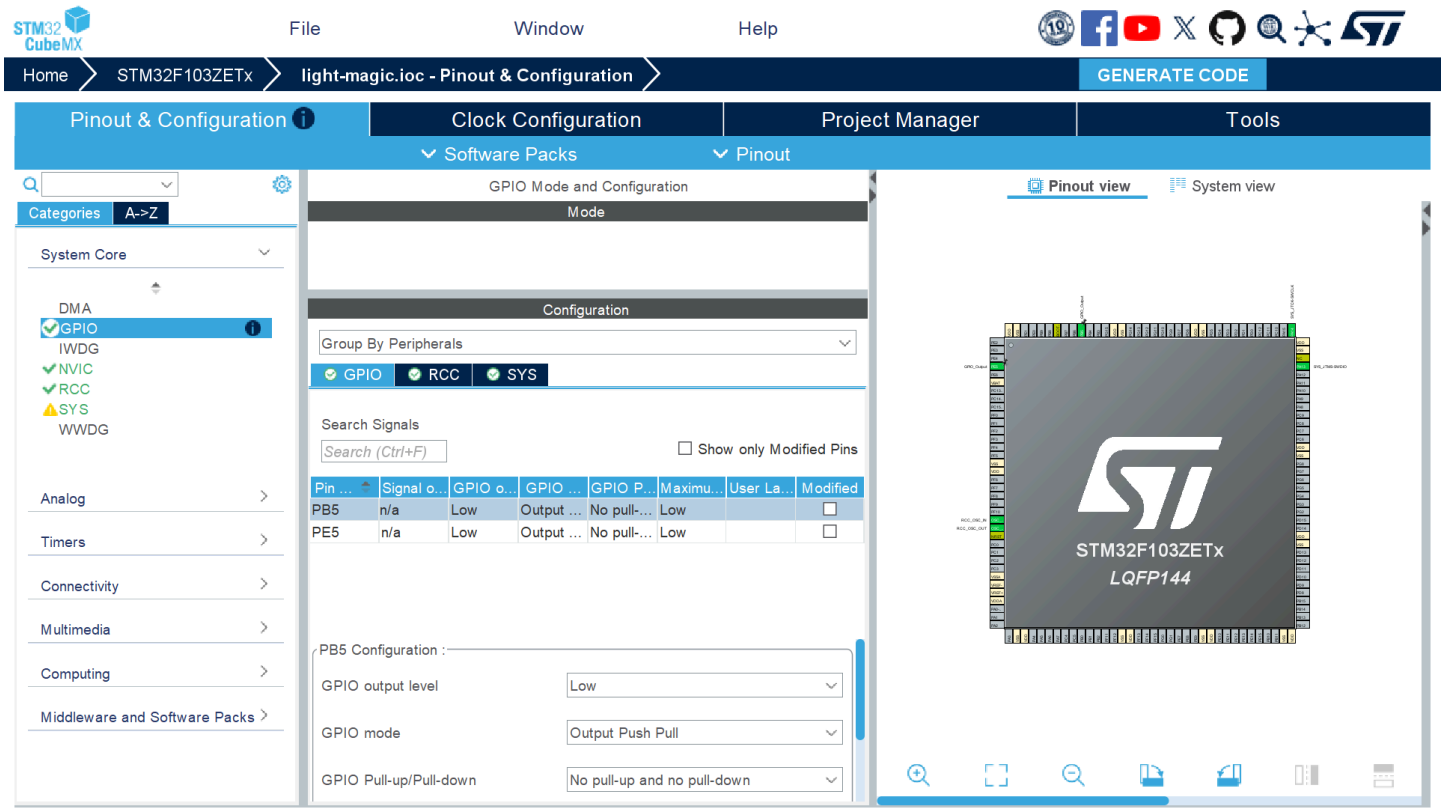


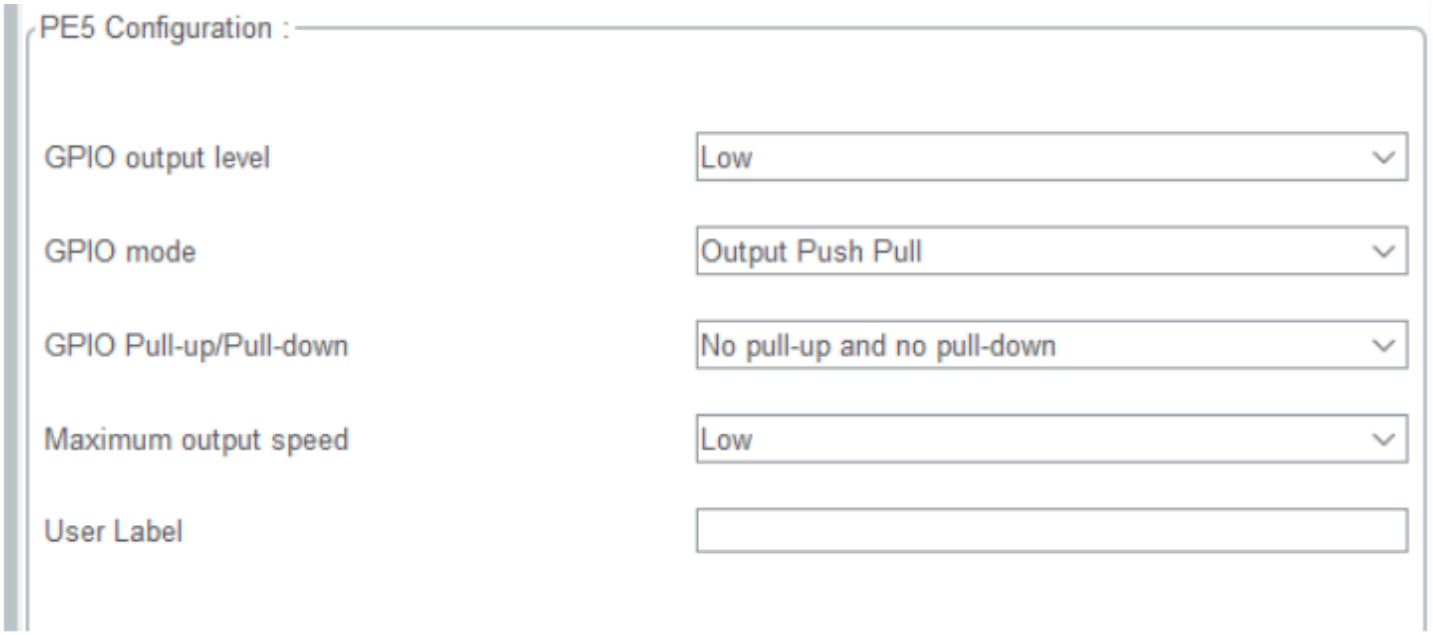
开发板的LED硬件连接原理图

还是这张图，可以看出控制LED的GPIO是PB5与PE5，将PB5与PE5配置为GPIO_Output



打开左侧栏GPIO选项





看下方的选项：

GPIO output level是初始时刻的电平。默认设置值为Low，那么LED的初始状态为点亮状态，**这里设置为High**，让LED初始处于熄灭状态。HAL库自动生成代码会使用HAL_GPIO_WritePin(GPIOE, GPIO_PIN_5, GPIO_PIN_RESET)来在一开始设置为低电平，如果选择了High参数就是GPIO_PIN_SET

GPIO mode就是GPIO模式，输出模式下是开漏（Output Open Drain）和推挽（OutPut Push Pull），这里选择**推挽输出**

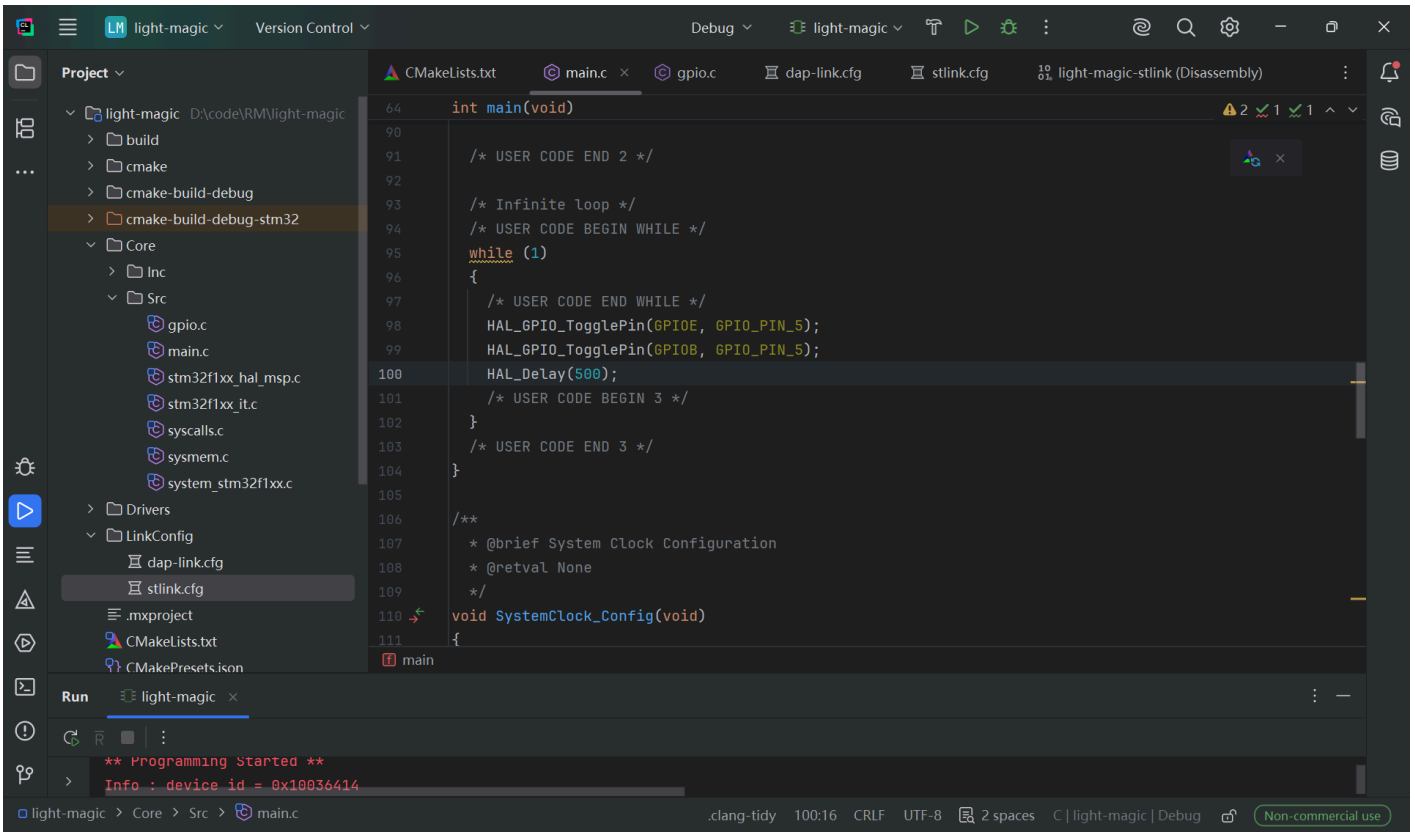
GPIO Pull-up/Pull-down是上拉还是下拉，由于本身就能稳定输出高 / 低电平，上拉下拉电阻不起作用，所以这里默认既不上拉也不下拉。

Maximum output speed是最大翻转速度，对于F1来说最高是50MHz。

User Label就是定义一个宏，代替GPIO_PIN_5或者其他引脚，可以让代码的可读性更强，这里写入PB5的User Lable 为LED0，PE5的User Lable 为LED1，与硬件电路对应。

接下来写按照通识的项目配置选项生成CMake工程。

CLion



mian()函数是所有函数的入口

打开工程在 main.c 第98行加入如下代码：

代码块

```
HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_5);
HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_5);
HAL_Delay(500);
```

或者如下代码

代码块

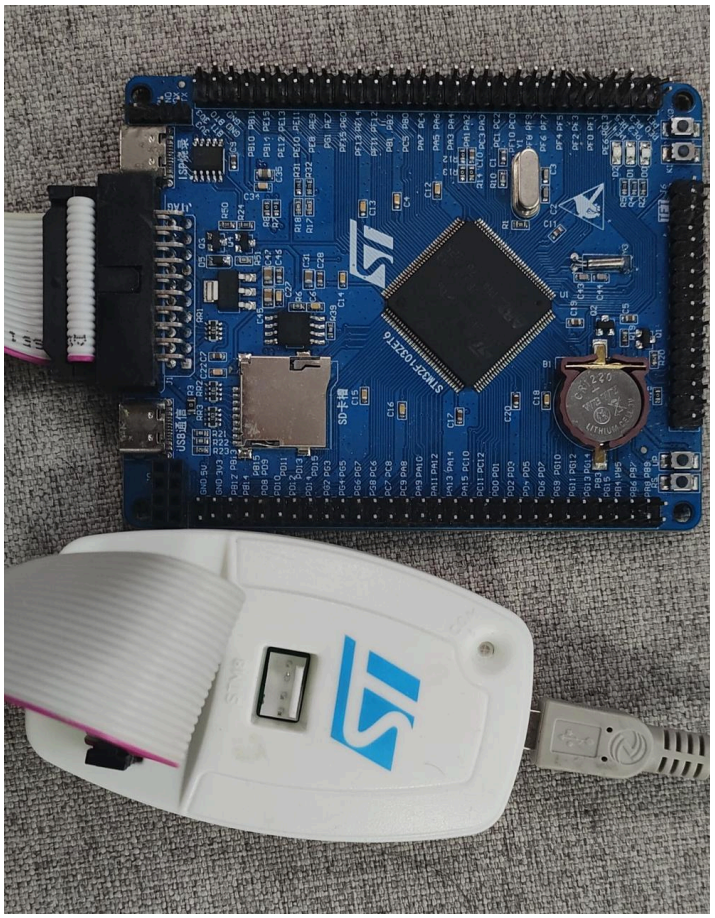
```
HAL_GPIO_TogglePin(LED1_GPIO_Port, LED1_Pin);
HAL_GPIO_TogglePin(LED0_GPIO_Port, LED0_Pin);
HAL_Delay(500);
```

转到LED0_GPIO_Port的定义(F12)，可以看到CubeMx的User Lable实际上是生成了如下宏定义

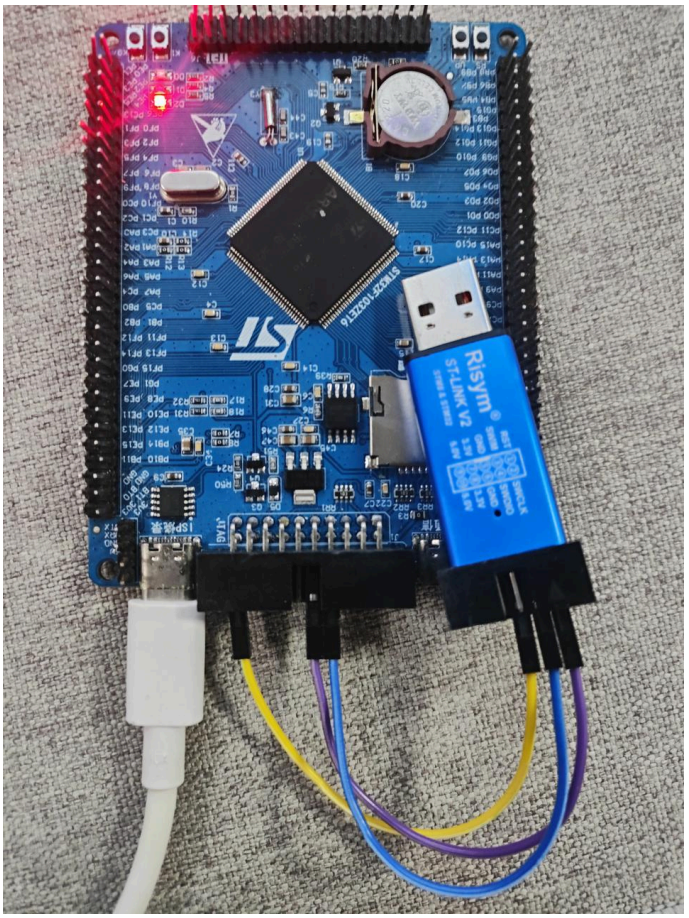
```
CMakeLists.txt  main.c  main.h  .gitignore  gpio.c  dap-link.cfg
53  void Error_Handler(void);
54
55  /* USER CODE BEGIN EFP */
56
57  /* USER CODE END EFP */
58
59  /* Private defines -----------------------------------------------------------*/
60  #define LED1_Pin GPIO_PIN_5
61  #define LED1_GPIO_Port GPIOE
62  #define LED0_Pin GPIO_PIN_5
63  #define LED0_GPIO_Port GPIOB
64
65  /* USER CODE BEGIN Private defines */
66
67  /* USER CODE END Private defines */
68
```

编译下载，先点小锤子(build)再点三角形(Run)

stlink需要安装驱动：https://www.st.com/zh/development-tools/stsw-link009.html/（如安装驱动仍不可用，看CLion开发进行驱动转换）



使用JTAG接口的STLink

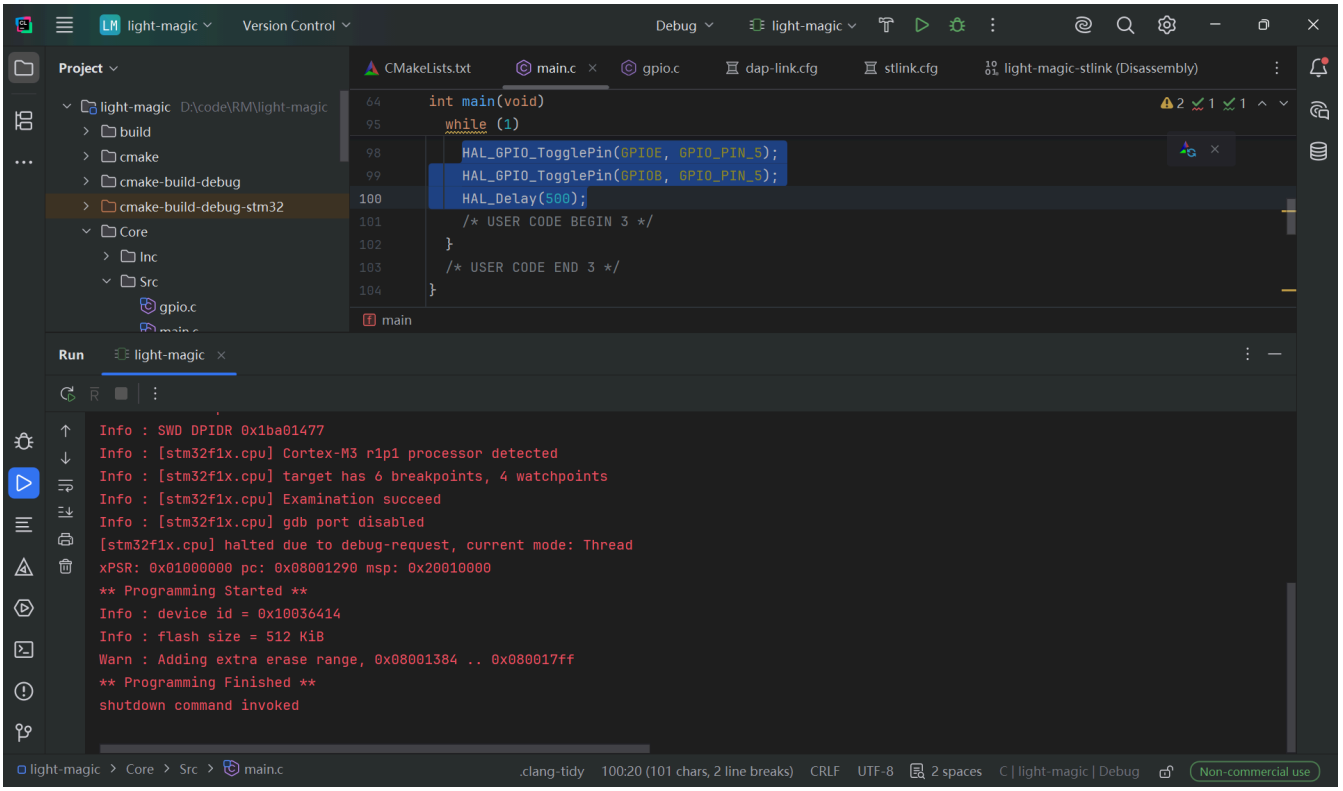


使用杜邦线有接错Vcc引脚的风险，这里建议新手使用USB-C接口进行供电

注意⚠️：其中板子出现未亮灯的情况，请使用**usb线进行供电**（如右图所示），建议**使用拓展坞**进行隔离（去年出现过电脑被烧的现象哦😨）

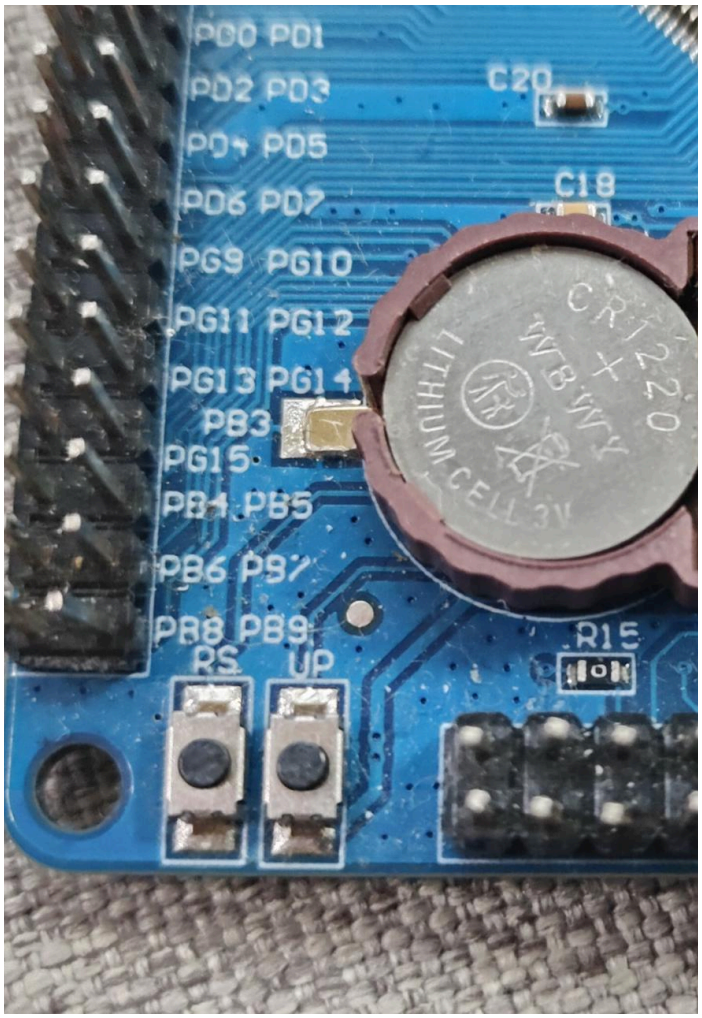
20针JTAG接口使用SWD协议接线（单片机一端）

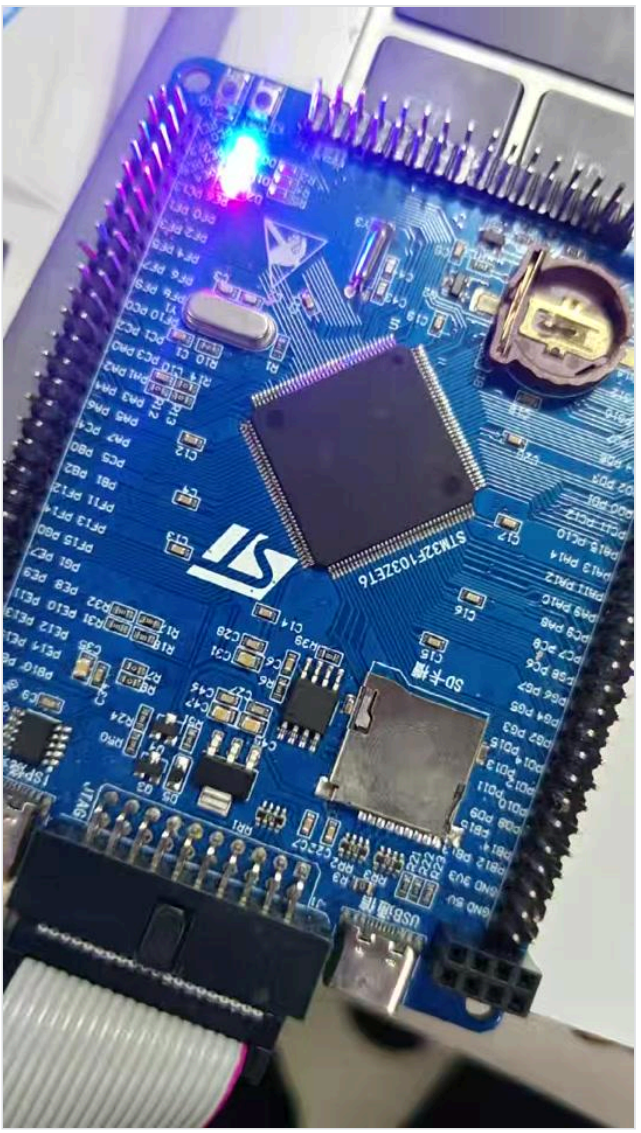
				豁口					
					SWCLK	SWDIO			**VCC(一般3v3，如使用type-C供电可不接)**
GND									



点灯魔法！

（红色灯是电源灯，不受程序控制）如果没有亮，可以先按一下reset按键（丝印是RS）试试





至此，我们已经完成了本章的最终目标。

接下来，再回到最开始的起点。重新审视刚才发生的一切。

四、GPIO的底层原理：寄存器与时钟

指针与地址

C语言之所以足够灵活，就是因为可以直观地操作内存，直接通过内存的地址值控制内存的内容。

在进行指针操作的时候，时刻记住自己是以CPU的身份进行操作的。

在单片机裸机程序中，指针存储的是真实物理地址，不存在操作系统的虚拟地址映射，程序可以通过指针直接访问统一编址的地址空间，包括 SRAM、Flash 以及外设寄存器。普通变量存放在 SRAM 中，使用指针读写属于常规内存操作；而外设寄存器拥有固定物理地址，将地址强制转为指针后解引用，就能直接配置硬件，这也是寄存器底层驱动的核心思路，示例如下：

代码块

```
1   volatile uint32_t *gpio_odr = (volatile uint32_t *)0x40020414;
```

```
2   *gpio_odr = 0x01;
```

这段代码通过指针操作 GPIOB 输出寄存器，令对应引脚输出高电平，`volatile` 用于阻止编译器优化，保证对寄存器的读写指令真实生效。

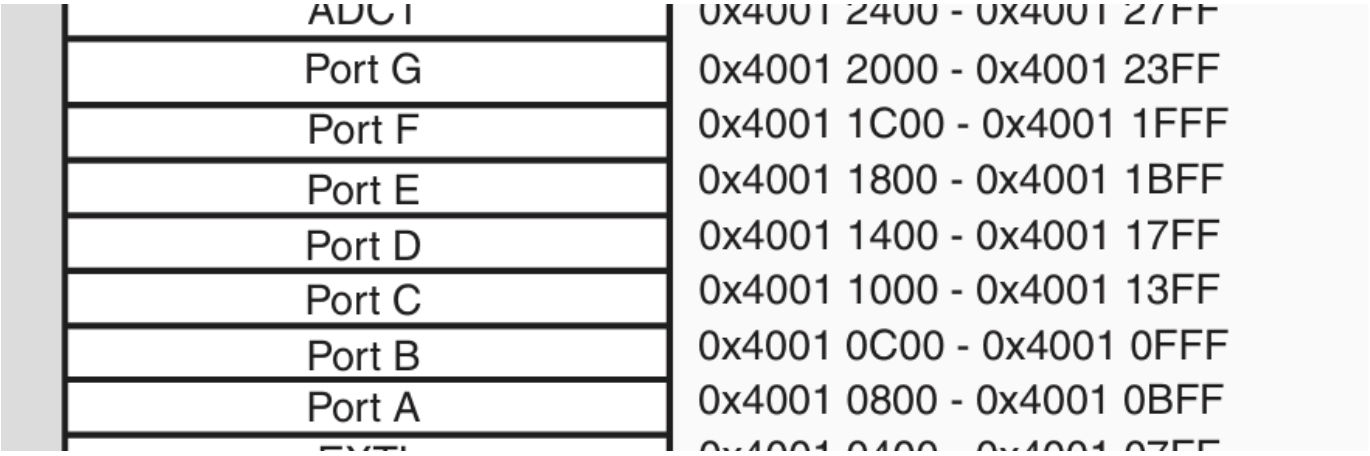
和 PC 端 C 程序不同，单片机没有内存保护机制，指针可以指向任意物理地址。如果指针越界、类型不匹配或者错误访问保留地址，不会触发系统异常提示，大概率直接引发 HardFault 死机，这也是嵌入式开发中使用指针最需要留意的风险。

GPIO寄存器定义与地址

前文提到过STM32系列MCU所有的输入输出行为都是通过访问0x4000 0000 - 0x5FFF FFFF这段神奇窗口实现的。这段窗口实际上就是STM32的外设寄存器的地址空间，STM32通过地址与外设寄存器产生数据交互，控制外设。

> 每个I/O端口位可以自由编程，然而必须按照32位字访问I/O端口寄存器(不允许半字或字节访问)。GPIOx_BSRR和GPIOx_BRR寄存器允许对任何GPIO寄存器进行读/更改的独立访问；这样，在读和更改访问之间产生IRQ时不会发生危险。

要控制GPIO引脚，就要知道它有哪些寄存器，这些寄存器具体在这段地址空间的什么位置，这些寄存器能够产生什么效果。下面使用STM32F103ZET6举例展示。(Port A代表GPIOA的一组16个引脚)



GPIO端口寄存器基地址

每个GPIO端口拥有的寄存器：

寄存器名称	寄存器描述	寄存器偏移地址	寄存器复位值
CRL	端口配置低寄存器	0x00	0x4444 4444
CRH	端口配置高寄存器	0x04	0x4444 4444

IDR	端口输入数据寄存器	0x08	0x0000 XXXX
ODR	端口输出数据寄存器	0x0C	0x0000 0000
BSRR	端口位设置/清除寄存器	0x10	0x0000 0000
BRR	端口位清除寄存器	0x14	0x0000 0000
LCKR	端口配置锁定寄存器	0x18	0x0000 0000

端口A-G都有各自的以上的寄存器，寄存器的地址为端口基地址加上偏移地址，复位值是默认值，X表示位不可用

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

CRL寄存器位定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

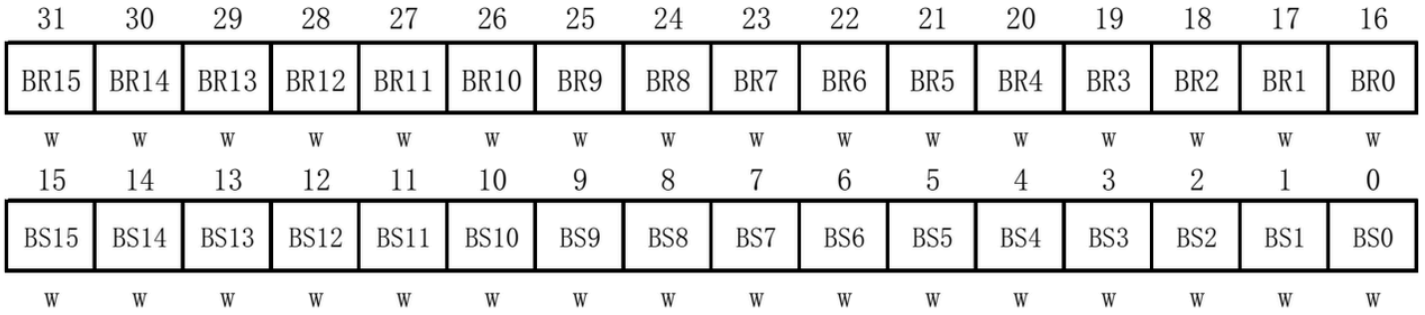
CRH寄存器位定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

IDR寄存器位定义



ODR寄存器位定义



BSRR寄存器位定义



LCKR寄存器位定义

总的来说地址分布表是这样的：

端口	基地址	寄存器	寄存器地址	位定义	偏移量	寄存器占用地址空间
GPIOA	0x4001 0800	CRL	0x4001 0800	CNF0-7, MODE0-7	0x00	0x04 Byte
		CRH	0x4001 0804	CNF8-15, MODE8-15	0X04	0X04 Byte
		IDR	0X4001 0808	IDR0-15	0X08	0X04 Byte
		ODR	0X4001 080C	ODR0-15	0x0C	0x04 Byte
		BSRR	0X4001 0810	BS0-15, BR0-15	0x10	0x04 Byte
		BRR	0X4001 0814	BR0-15	0x14	0x04 Byte

		LCKR	0X4001 0818	LCK0-15, LCKK	0x18	0x04 Byte
GPIOB	0x4001 0C00	CRL	0x4001 0C00	CNF0-7, MODE0-7	0x00	0x04 Byte
		CRH	0x4001 0C04	CNF8-15, MODE8-15	0X04	0X04 Byte
		IDR	0X4001 0C08	IDR0-15	0X08	0X04 Byte
		ODR	0X4001 0C0C	ODR0-15	0x0C	0x04 Byte
		BSRR	0X4001 0C10	BS0-15, BR0-15	0x10	0x04 Byte
		BRR	0X4001 0C14	BR0-15	0x14	0x04 Byte
		LCKR	0X4001 0C18	LCK0-15, LCKK	0x18	0x04 Byte
GPIOC	0x4001 1000	CRL	0x4001 1000	CNF0-7, MODE0-7	0x00	0x04 Byte
		CRH	0x4001 1004	CNF8-15, MODE8-15	0X04	0X04 Byte
		IDR	0X4001 1008	IDR0-15	0X08	0X04 Byte
		ODR	0X4001 100C	ODR0-15	0x0C	0x04 Byte
		BSRR	0X4001 1010	BS0-15, BR0-15	0x10	0x04 Byte
		BRR	0X4001 1014	BR0-15	0x14	0x04 Byte
		LCKR	0X4001 1018	LCK0-15, LCKK	0x18	0x04 Byte
GPIOD	0x4001 1400	CRL	0x4001 1400	CNF0-7, MODE0-7	0x00	0x04 Byte
		CRH	0x4001 1404	CNF8-15, MODE8-15	0X04	0X04 Byte
		IDR	0X4001 1408	IDR0-15	0X08	0X04 Byte
		ODR	0X4001 140C	ODR0-15	0x0C	0x04 Byte
		BSRR	0X4001 1410	BS0-15, BR0-15	0x10	0x04 Byte
		BRR	0X4001 1414	BR0-15	0x14	0x04 Byte
		LCKR	0X4001 1418	LCK0-15, LCKK	0x18	0x04 Byte

GPIOE	0x4001 1800	CRL	0x4001 1800	CNF0-7, MODE0-7	0x00	0x04 Byte
		CRH	0x4001 1804	CNF8-15, MODE8-15	0X04	0X04 Byte
		IDR	0X4001 1808	IDR0-15	0X08	0X04 Byte
		ODR	0X4001 180C	ODR0-15	0x0C	0x04 Byte
		BSRR	0X4001 1810	BS0-15, BR0-15	0x10	0x04 Byte
		BRR	0X4001 1814	BR0-15	0x14	0x04 Byte
		LCKR	0X4001 1818	LCK0-15, LCKK	0x18	0x04 Byte
GPIOF	0x4001 1C00	CRL	0x4001 1C00	CNF0-7, MODE0-7	0x00	0x04 Byte
		CRH	0x4001 1C04	CNF8-15, MODE8-15	0X04	0X04 Byte
		IDR	0X4001 1C08	IDR0-15	0X08	0X04 Byte
		ODR	0X4001 1C0C	ODR0-15	0x0C	0x04 Byte
		BSRR	0X4001 1C10	BS0-15, BR0-15	0x10	0x04 Byte
		BRR	0X4001 1C14	BR0-15	0x14	0x04 Byte
		LCKR	0X4001 1C18	LCK0-15, LCKK	0x18	0x04 Byte
GPIOG	0x4001 2000	CRL	0x4001 2000	CNF0-7, MODE0-7	0x00	0x04 Byte
		CRH	0x4001 2004	CNF8-15, MODE8-15	0X04	0X04 Byte
		IDR	0X4001 2008	IDR0-15	0X08	0X04 Byte
		ODR	0X4001 200C	ODR0-15	0x0C	0x04 Byte
		BSRR	0X4001 2010	BS0-15, BR0-15	0x10	0x04 Byte
		BRR	0X4001 2014	BR0-15	0x14	0x04 Byte
		LCKR	0X4001 2018	LCK0-15, LCKK	0x18	0x04 Byte

GPIO的使用

✍️ 在启用GPIO功能之前往往需要先使能GPIO端口的时钟，但是时钟的使能在CubeMX中配置后自动完成，因此不特殊强调，时钟默认使能。

单独的位设置或位清除

> 当对GPIOx_ODR的个别位编程时，软件不需要禁止中断：在单次APB2写操作里，可以只更改一个或多个位。
>
> 这是通过对“置位/复位寄存器”(GPIOx_BSRR，复位是 GPIOx_BRR)中想要更改的位写’1’来实现的。没被选择的位将不被更改。

这段话的意思是我们之前通过给ODR进行赋值的时候，我们只想改变某个bit而不想改变其他bit的时候，肯定需要先读取ODR的值，保持其他bit的值不变，只把这bit的值清空。但是这样读取后再复制需要两步，不够迅捷。比如我们只想让PF5从高电平拉低，那么我们需要读取GPIOF的ODR寄存器的值，假设为0x00FF(1111 1111b)，然后只把bit5设置为0，其他的bit保持不变，这样我们应该给ODR重新赋值为0x00EF(1110 1111b)，如果使用BSRR，我们只需要向BRR写入0x001F(0001 0000b)只把我们需要更改的bit5设置为1，硬件就会帮我们保留其他为，只把bit5清0，这样操作不需要读取任何寄存器。反而，如果想要拉高，可以向BSRR写入0x001F。

GPIO锁定

GPIO 锁定通过 GPIOx_LCKR 寄存器实现，可以冻结指定 GPIO 引脚的模式、上下拉、速率等配置信息。锁定生效后，软件无法再修改这些配置，只有单片机复位才能解除锁定，一般用来保护晶振、硬件使能这类关键引脚，避免程序跑飞、异常逻辑意外改动 IO 配置。该锁定需要遵循一套固定的写时序才能成功启用，**只锁引脚配置，不锁引脚的输出电平**，锁定后依然可以正常读写引脚高低电平。

代码块

```
// HAL库示例，锁定PB0引脚
HAL_GPIO_LockPin(GPIOB, GPIO_PIN_0);
```

输入配置

当I/O端口配置为输入时：

- 输出缓冲器被禁止
- 施密特触发输入被激活
- 根据输入配置(上拉，下拉或浮动)的不同，弱上拉和下拉电阻被连接
- 出现在I/O脚上的数据在每个APB2时钟被采样到输入数据寄存器
- 对输入数据寄存器的读访问可得到I/O状态

输出配置

当I/O端口被配置为输出时：

● 输出缓冲器被激活

— 开漏模式：输出寄存器上的’0’激活N-MOS，而输出寄存器上的’1’将端口置于高阻状态(PMOS从不被激活)。

— 推挽模式：输出寄存器上的’0’激活N-MOS，而输出寄存器上的’1’将激活P-MOS。

● 施密特触发输入被激活

● 弱上拉和下拉电阻被禁止

● 出现在I/O脚上的数据在每个APB2时钟被采样到输入数据寄存器

● 在开漏模式时，对输入数据寄存器的读访问可得到I/O状态

● 在推挽式模式时，对输出数据寄存器的读访问得到最后一次写的值。

配置模式

> 当作为输出配置时，写到输出数据寄存器上的值(GPIOx_ODR)输出到相应的I/O引脚。可以以推挽模式或开漏模式(当输出0时，只有N-MOS被打开)使用输出驱动器。
>
> 输入数据寄存器(GPIOx_IDR)在每个APB2时钟周期捕捉I/O引脚上的数据。所有GPIO引脚有一个内部弱上拉和弱下拉，当配置为输入时，它们可以被激活也可以被断开。

从GPIO的所有模式中选取一种，根据表格

MODE[1:0]	意义
00	保留
01	最大输出速度为10MHz
10	最大输出速度为2MHz
11	最大输出速度为50MHz

配置模式		CNF1	CNF0	MODE1	MODE0	PxODR寄存器
通用输出	推挽(Push-Pull)	0	0	01 10 11 见表18		0 或 1
	开漏(Open-Drain)		1			0 或 1
复用功能输出	推挽(Push-Pull)	1	0			不使用
	开漏(Open-Drain)		1			不使用
输入	模拟输入	0	0	00		不使用
	浮空输入		1			不使用
	下拉输入	1	0			0
	上拉输入					1

记录下**CNF位**应该是0还是1，**MODE位**应该是0还是1，**ODR寄存器**应该设置为多少。

例如，我要将PE5引脚配置为通用推挽输出,最高速率，并且拉高，应该将GPIOE这个外设的CRL这个寄存器的CNF5这个位置设置为00,MODE5设置为11,ODR寄存器的bit5配置为1。

首先找到GPIOD的基地址为0x4001 1800，CRL的偏移地址为0x00，因此GPIOD的CRL的地址为0x4001 1800，然后找到CNF5为CRL的第22位和第23位，将其配置为00，在找到MODE5为CRL的第20位和21位，都设置为1，找到ODR寄存器地址为0x4001 1800 + 0x0C为0x4001 180C，将其第5位设置为1。

使用C写一个函数如下：

拉高PE5

```c
void setThePE5(void) {
    uint32_t* GPIOE_CRL = (uint32_t*)0x40011800;
    uint32_t* GPIOE_ODR = (uint32_t*)0x4001180C;

    *GPIOE_CRL &= ~((1 << 22) | ( 1 << 23));
    *GPIOE_CRL |= (1 << 20) | (1 << 21);
    *GPIOE_ODR |= 1 << 5;
}
```

配置输入或者输出

如果你启用了输出模式，可以控制ODR或者BSRR进行GPIO的控制。

这里再次说明一下使用这两个寄存器有什么区别：

如果你使用**ODR**控制GPIO的电平高低，你很可能要先读取**ODR**的值，因为如果你只想改变某个引脚的电平，那么你就只能动**ODR**对应bit的0或者1，保持其他的bit不变，但是你要赋值的时候只能给**ODR**整体赋值，因此你要先读取**ODR**原来的值，再将原来的值得某一个bit修改。所以要先读取，后写入。

如果你使用**BSRR**控制GPIO，无需读取**ODR**的值，你向**BSRR**写入了第几位为1，**BSRR**就会帮你只修改对应的位置。低16位负责将**ODR**拉高，高16位负责将**ODR**拉低。

如果你启用了输入模式，模拟和浮空输入无需配置**ODR**，只需要读取**IDR**

五、ST提供的HAL库

在使用外设的时候需要繁琐地与寄存器打交道，为了方便开发者开发自家芯片，ST推出了HAL库，将与寄存器的配置等打包成函数，直接调用此函数，就相当于完成了寄存器的配置。

当你使用CubeMX自动生成工程的时候，默认使用HAL库。

ST提供的HAL库中stm32f1xx_hal_gpio.c中提供了控制GPIO的函数，以及上面用到的HAL_Delay()：

函数名	功能	参数
HAL_GPIO_Init(GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *GPIO_Init)	初始化GPIO端口和引脚	GPIO结构体， GPIO初始化信息结构体
HAL_GPIO_DeInit(GPIO_TypeDef *GPIOx, uint32_t GPIO_Pin)	反初始化GPIO	GPIO结构体，引脚
HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)	读取引脚电平	GPIO结构体，引脚
HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)	控制引脚电平	GPIO结构体，引脚，电平高低
HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)	反转引脚电平	GPIO结构体，引脚
HAL_GPIO_LockPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)	锁定引脚电平	GPIO结构体，引脚
HAL_Delay(uint32_t ms)	程序会阻塞等待指定毫秒时间， 期间 CPU 不会执行后续代码	需要阻塞延时的时间

> 补充说明：
>
> 1. `GPIO_PinState` 枚举：`GPIO_PIN_RESET`(低)、`GPIO_PIN_SET`(高)
> 2. `HAL_GPIO_LockPin` 返回`HAL_StatusTypeDef`，锁定成功返回`HAL_OK`；锁定的是**引脚配置**，不是电平

> 3. `GPIO_TypeDef` 是端口寄存器结构体，如`GPIOA`、`GPIOB`
>
> 此外还提供了宏函数：__HAL_RCC_GPIOx_CLK_ENABLE()用于使能GPIO端口时钟

HAL_GPIO_Init与HAL_GPIO_DeInit就是通过配置CRL和CRH寄存器实现配置GPIO设置

HAL_GPIO_ReadPin通过读取IDR寄存器读取

HAL_GPIO_WritePin和HAL_GPIO_TogglePin通过BSRR寄存器完成GPIO的拉高和拉低

HAL_GPIO_LockPin通过LCKR寄存器完成GPIO锁定

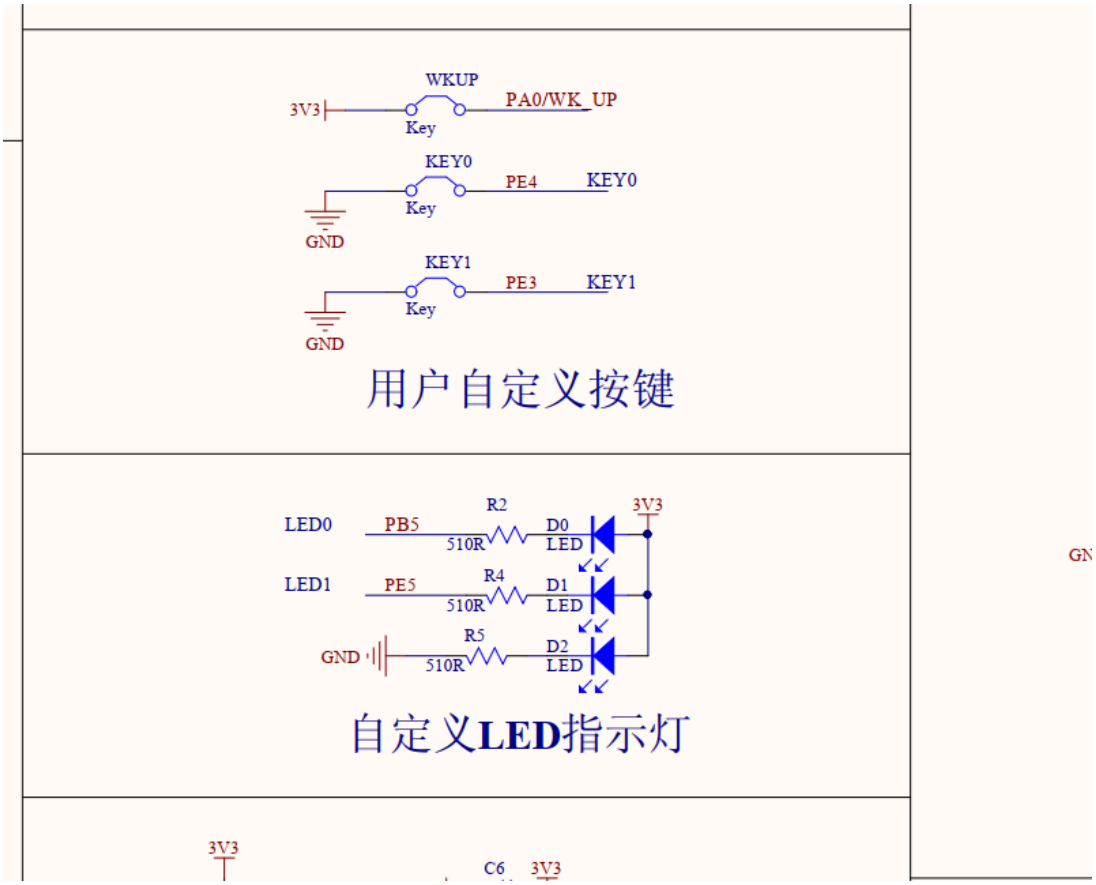
⚔️-二周目

在了解了GPIO与C语言语法之后，我们又回到了最开始的章节，再次点亮一颗LED，但是这一次，我们对于单片机工程有着不一样的理解。首先我们拥有了一定的C语言知识，可以看得懂一部分代码了，其次我们了解了GPIO的构成，能够明白代码如何控制LED亮灭。

🚀高速咏唱

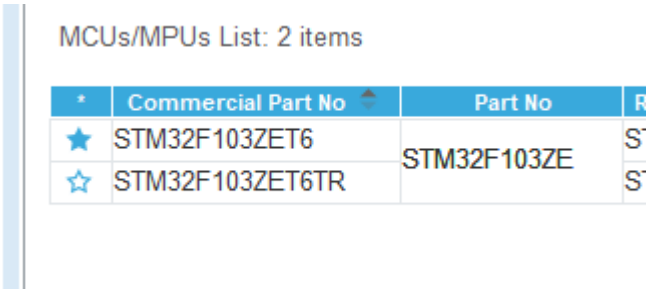
使用HAL库函数可以很方便地控制GPIO，但是实际上HAL库函数中没有很复杂的部分,而且还会有函数调用的性能开销，只是简单地通过BSRR写入引脚电平和通过IDR读取引脚电平。因此我们舍弃HAL库函数，使用更加直接更加底层的方式直接控制GPIO，避免函数调用带来的性能消耗。我将这种控制GPIO的方式称为**高速咏唱**😋（

硬件连接



时钟总线

新建CubeMX工程，选择STM32F103ZET6



配置SYS中Debug为Serial Wire，RCC中HSE配置为晶振，时钟树中主频为8MHz

工程命名为High-Speed-Light-Magic，GENERATE CODE，选择CMake，打开项目

使能APB2上GPIOE与GPIOB时钟

```
88  MX_GPIO_Init();
89  /* USER CODE BEGIN 2 */
90  __HAL_RCC_GPIOE_CLK_ENABLE();
91  __HAL_RCC_GPIOB_CLK_ENABLE();
92  /* USER CODE END 2 */
93
94  /* Infinite loop */
```

接着往下写

```
int main(void)
  /* USER CODE BEGIN 2 */
  __HAL_RCC_GPIOE_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  #define CRL_OFFSET 0x00
  #define ODR_OFFSET 0x0C
  #define BSRR_OFFSET 0x10

  uint32_t pEBase = 0x40011800; // 存储GPIOE基地址
  uint32_t pBBase = 0x40010C00; // 存储GPIOB基地址

  uint32_t * pECrl = (uint32_t *)(pEBase + CRL_OFFSET); // 使用指针存储GPIOE CRL的寄存器地址
  uint32_t * pEOdr = (uint32_t *)(pEBase + ODR_OFFSET); // 使用指针存储GPIOE ODR的寄存器地址
  uint32_t * pEBSrr = (uint32_t *)(pEBase + BSRR_OFFSET); // 使用指针存储GPIOE BSRR的寄存器地址

  uint32_t * pBCrl = (uint32_t *)(pBBase + CRL_OFFSET);
  uint32_t * pBOdr = (uint32_t *)(pBBase + ODR_OFFSET);
  uint32_t * pBBSrr = (uint32_t *)(pBBase + BSRR_OFFSET);

  *pECrl &= ~((1 << 22) | (1 << 23)); // 配置GPIOE的CRL寄存器，推挽输出
  *pECrl |= (1 << 20) | (1 << 21);

  *pBCrl &= ~((1 << 22) | (1 << 23)); // 配置GPIOB的CRL寄存器，推挽输出
  *pBCrl |= (1 << 20) | (1 << 21);

  *pEBSrr |= 1 << (5 + 16); // 拉低PE5，点亮LED1
  *pBBSrr |= 1 << 5;        // 拉高PB5，熄灭LED0
```

翻转引脚

这里使用for循环阻塞CPU形成延时。使用变量暂存灯的状态，不需要每次读取ODR即可完成翻转

```
119   /* USER CODE BEGIN WHILE */
120   while (1)
121   {
122     static uint8_t state;
123     *pEBSrr |= 1 << (5 + 16 * state);
124     *pBBSrr |= 1 << (5 + 16 * (!state));
125     state = !state;
126     for(volatile uint16_t i = 1; i != 0; i++);
127     /* USER CODE END WHILE */
```

这里给BSRR的bit5置1可以只控制引脚5拉高，不会影响其他引脚的状态。而给bit(5 + 16)即bit21置1的话则是只拉低引脚5，因此当state为0的时候，GPIOE的BSRR寄存器的bit5被置一，而GPIOB的BSRR寄存器的bit21会被置一。如果state等于1的话则是反过来。

使用static变量能够保持每次进入函数或者while语句时变量值不被重置，使用volatile防止语句被编译器优化舍弃。

向凶险之地迈出第一步

在拥有了GPIO点灯底层操作的经验之后，接下来我们来分析HAL库能够使用的原理。(我打HAL库？真的假的?

这里分析HAL库初始化刚刚配置的PE5的代码

代码块

```
1
2   /**
3     * @brief GPIO Initialization Function
4     * @param None
5     * @retval None
6     */
7   static void MX_GPIO_Init(void)
8   {
9     GPIO_InitTypeDef GPIO_InitStruct = {0};
10    /* USER CODE BEGIN MX_GPIO_Init_1 */
11
12    /* USER CODE END MX_GPIO_Init_1 */
13
14    /* GPIO Ports Clock Enable */
15    __HAL_RCC_GPIOE_CLK_ENABLE();
16    __HAL_RCC_GPIOA_CLK_ENABLE();
17
18    /*Configure GPIO pin Output Level */
19    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_5, GPIO_PIN_RESET);
20
21    /*Configure GPIO pin : PE5 */
22    GPIO_InitStruct.Pin = GPIO_PIN_5;
23    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
24    GPIO_InitStruct.Pull = GPIO_NOPULL;
25    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
26    HAL_GPIO_Init(GPIOE, &GPIO_InitStruct);
27
28    /* USER CODE BEGIN MX_GPIO_Init_2 */
29
30    /* USER CODE END MX_GPIO_Init_2 */
31  }
```

GPIO_InitTypeDef是GPIO初始化配置结构体，不只是GPIO，所有的外设初始化都需要将初始化配置写入结构体中，然后将结构体传入函数，完成配置。这一步实际上就是在完成配置GPIOx的CRL和CRH寄存器

GPIO_InitTypeDef类型的定义如下：

代码块

```
/**
  * @brief GPIO Init structure definition
  */
typedef struct
{
  uint32_t Pin;       /*!< Specifies the GPIO pins to be configured.
                           This parameter can be any value of @ref GPIO_pins_define */

  uint32_t Mode;      /*!< Specifies the operating mode for the selected pins.
                           This parameter can be a value of @ref GPIO_mode_define */

  uint32_t Pull;      /*!< Specifies the Pull-up or Pull-Down activation for the selected pins.
                           This parameter can be a value of @ref GPIO_pull_define */

  uint32_t Speed;     /*!< Specifies the speed for the selected pins.
                           This parameter can be a value of @ref GPIO_speed_define */
} GPIO_InitTypeDef;
```

1. 面对的第一个函数：

__HAL_RCC_GPIOE_CLK_ENABLE

在Keil编译之后，将光标插入函数名或者变量名，按F12跳转到定义位置

代码块

```
#define __HAL_RCC_GPIOE_CLK_ENABLE()   do { \
                                          __IO uint32_t tmpreg; \
                                          SET_BIT(RCC->APB2ENR, RCC_APB2ENR_IOPEEN);\
                                          /* Delay after an RCC peripheral clock enabling */ \
                                          tmpreg = READ_BIT(RCC->APB2ENR, RCC_APB2ENR_IOPEEN);\
```

```
6                                                   UNUSED(tmpreg); \
7                                                 } while(0U)
```

这段代码有一个很明显的标志：**#define**，这表示__HAL_RCC_GPIOE_CLK_ENABLE这个函数是一个宏

宏就是替换，因此*__HAL_RCC_GPIOE_CLK_ENABLE*这段文字会被替换成下面的代码

代码块

```
1   do {
2       __IO uint32_t tmpreg;
3       SET_BIT(RCC->APB2ENR, RCC_APB2ENR_IOPEEN);
4       /* Delay after an RCC peripheral clock enabling */
5       tmpreg = READ_BIT(RCC->APB2ENR, RCC_APB2ENR_IOPEEN);
6       UNUSED(tmpreg);
7   } while(0U)
```

do{}while()语句与while()的区别在于while()语句是先判断，后执行，do{}while()是先执行，后判断。这里while里的条件是0，意思是这段代码只会执行一次，这么写是因为宏是不会检查语法的单纯替换，为了防止替换之后出现其他问题，使用do{}while()语句将这段代码与其他代码隔开，更加安全。

这里注意一下，以及自己写宏的时候也要注意，定义宏的时候是不能带最后一个结尾的;的。除非你在调用宏的时候不写;，那这样写出来的代码看起来不符合规范。因此写宏的时候不要带最后一个分号，调用宏的时候在宏的末尾加上那个分号。

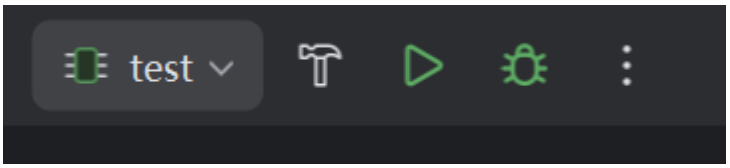
__IO, SET_BIT, READ_BIT, UNUSED，RCC_APB2ENR_IOPEEN都是宏。具体用处这里不细说，有兴趣可以自己看一看定义。

这里的作用就是将**RCC**的**APB2ENR**寄存器的**IOPEEN**位置一，使能GPIOE的时钟

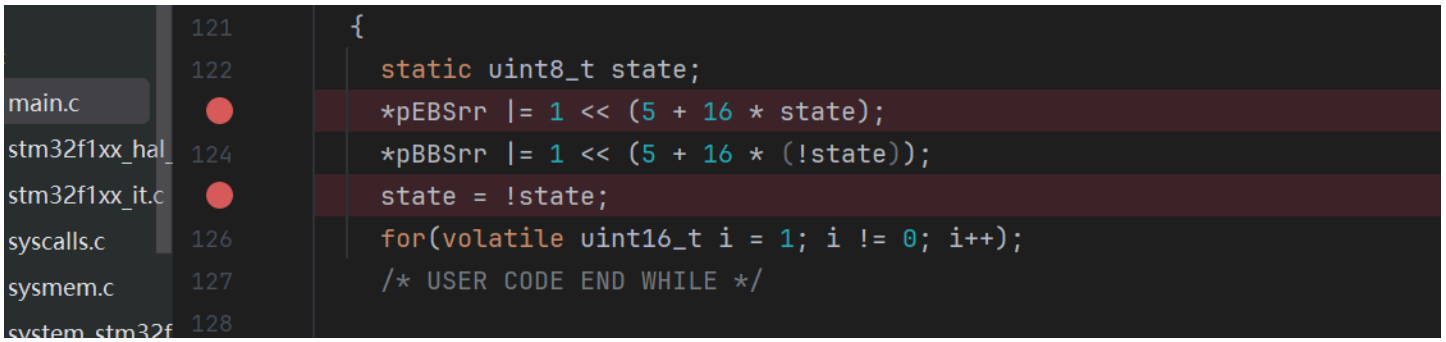
六、Debug

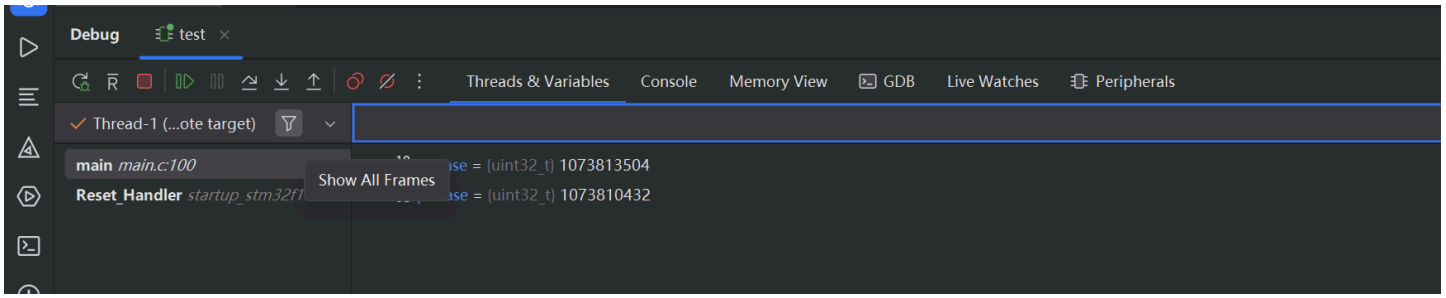
当代码和预想不一样时，Debug 可以让你亲眼看见程序到底怎么运行，找到 bug 在哪。

点击最右侧甲虫图标进入Debug功能



在行号处可以打断点，程序运行到断点





从左向右的功能是：

🔄 小虫子图标**重启并调试（Reset & Run）**复位单片机，从头开始跑程序，自动进入调试模式

R **软复位 MCU（Reset）**单片机复位，程序回到 `Reset_Handler` 从头执行，**不重新下载程序**

🟥 方框**停止调试（Disconnect）**暂停运行，退出调试会话，断开和芯片的连接

▶ 继续运行（Resume）断点停下后，全速跑，直到下一个断点

⏸ 暂停（Pause）程序正在跑的时候，立刻暂停当前代码

↰ **Step Over（F10，单步跳过）**执行**当前整行代码**，如果这一行是函数，不会跳进函数内部

↓ **Step Into（F11，单步进入）**如果当前行是函数，**跳进函数内部**一行一行跑

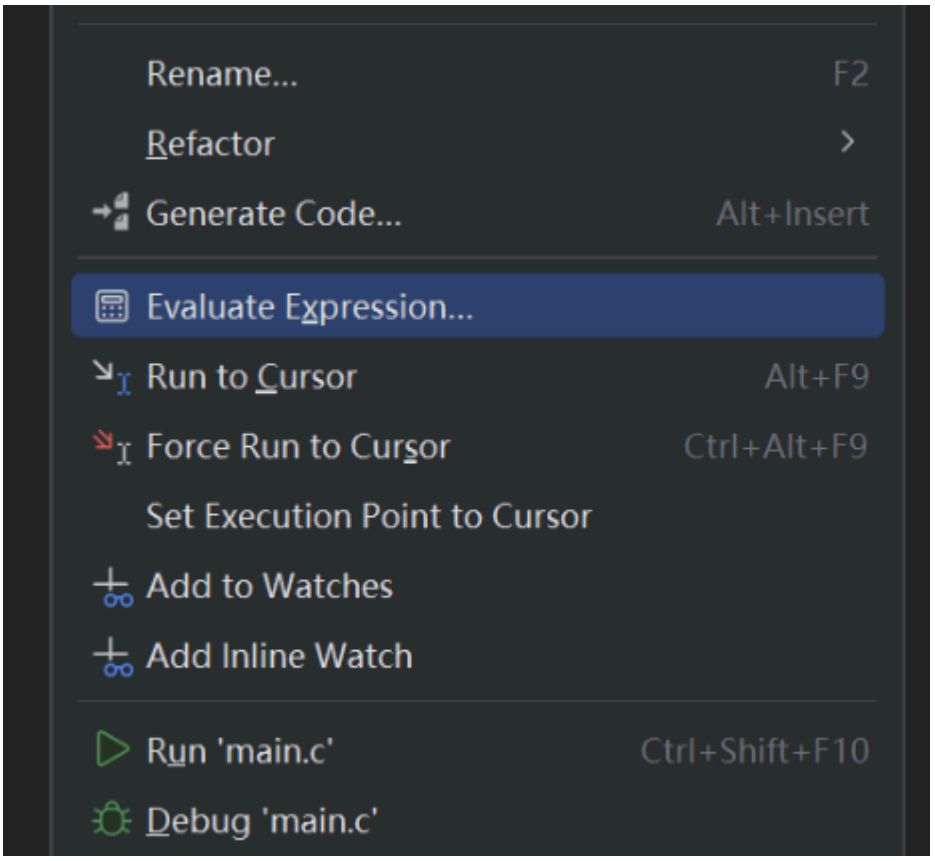
↑ **Step Out（Shift+F11，单步跳出）**快速跑完当前剩下的函数代码，直接跳出回到上层

● 添加 / 移除断点在代码行打停止标记

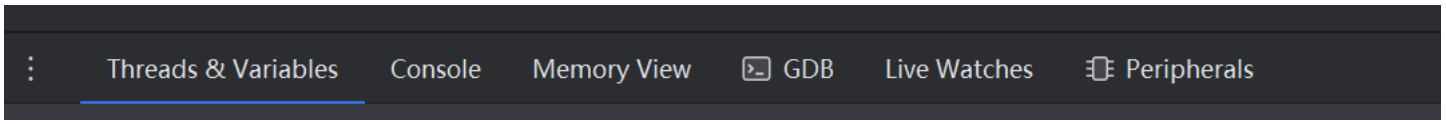
⭕禁用断点断点暂时失效，不用删掉

右击鼠标，从现在光标处开始，依次往下是：

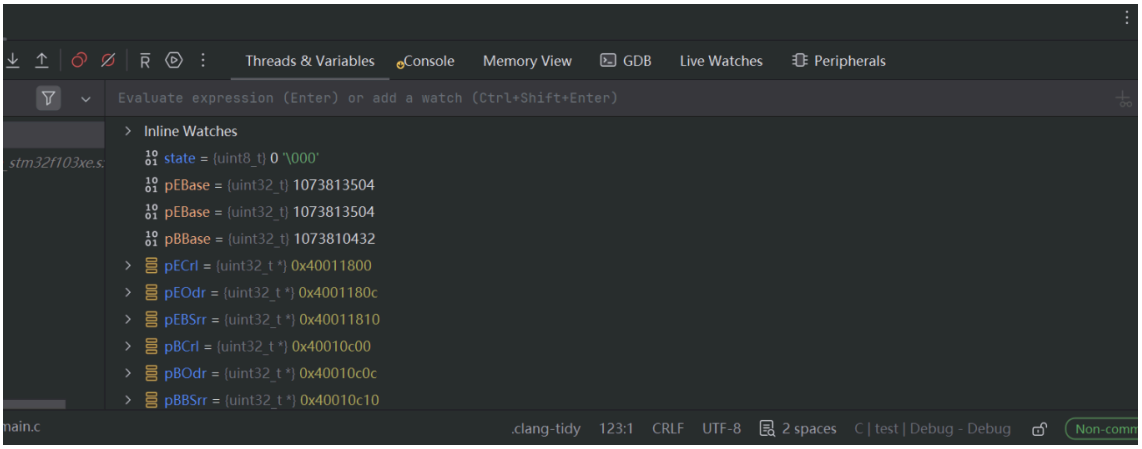
- 表达式求值，可以临时输入变量、寄存器、运算式，实时算出当前值，也可以临时修改变量的值
- 运行到光标处：中间代码自动跑完，遇到断点也会停；
- 强制运行到光标处：强制跑到光标处，**跳过中途所有断点**，只在光标位置停下；
- 跳转到光标处（很少使用）：**不会执行中间代码**，直接来到当前语句，相当于强行修改下一步要跑的代码；
- 添加到监视：将变量添加到监视（只有全局变量才能添加），监视只有暂停程序才会更新
- 添加到内联监视



右侧面板：



Threads & Variables：查看当前线程、实时变量值，用于程序暂停时，看栈、局部变量，添加到inline watches的值也在这里查看



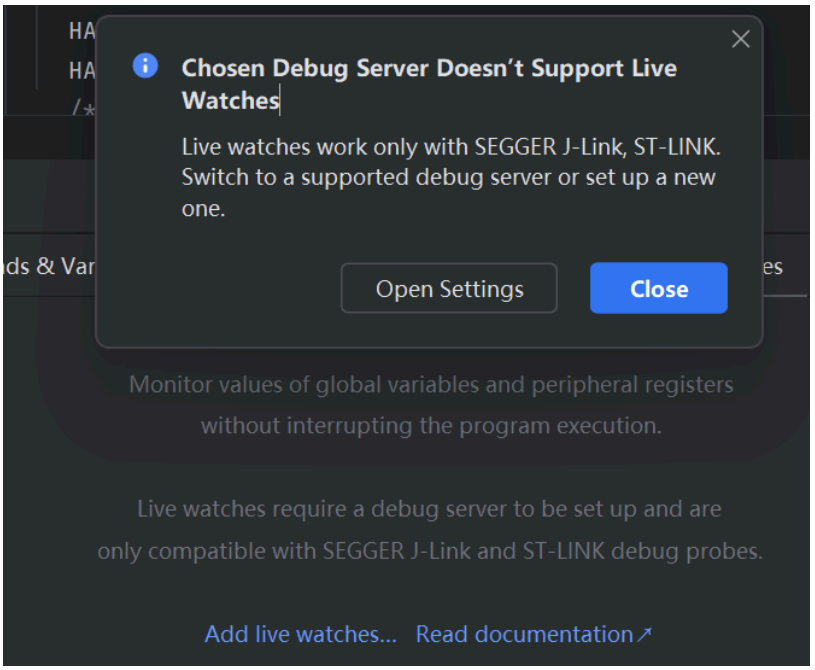
Console（调试控制台 / 串口控制台）：用来输出单片机程序里主动打印出来的文字信息（比如`printf`）

Memory View：最底层，按内存地址看原始字节，不受变量、外设限制

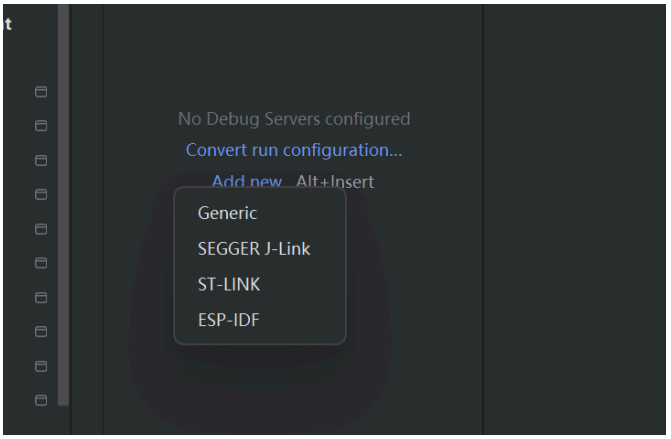
GDB：记录调试底层日志，图形界面所有暂停、断点动作，背后都会在这里留下记录

Live Watches：单片机全速跑的时候，自动持续刷新你指定的变量 / 表达式，不用暂停程序

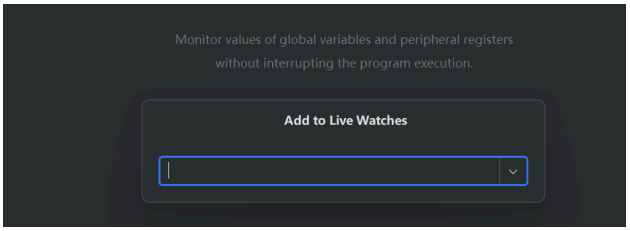
Add live Watches，出现上述弹窗点击Open Settings



Add new，根据你的调试器对应选择，然后直接Apply即可

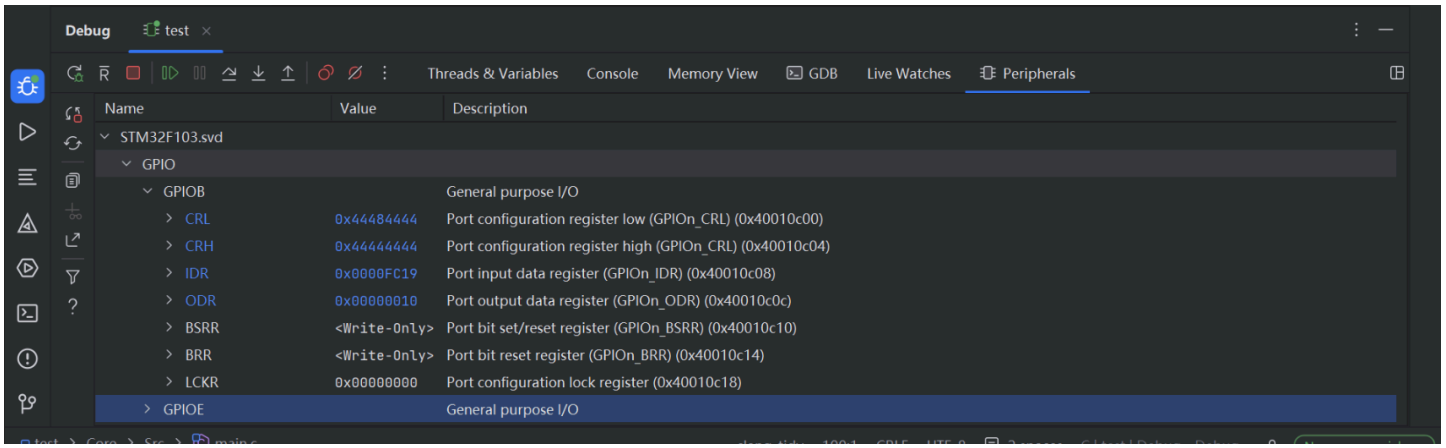


现在就可以输入你想实时监视的值了

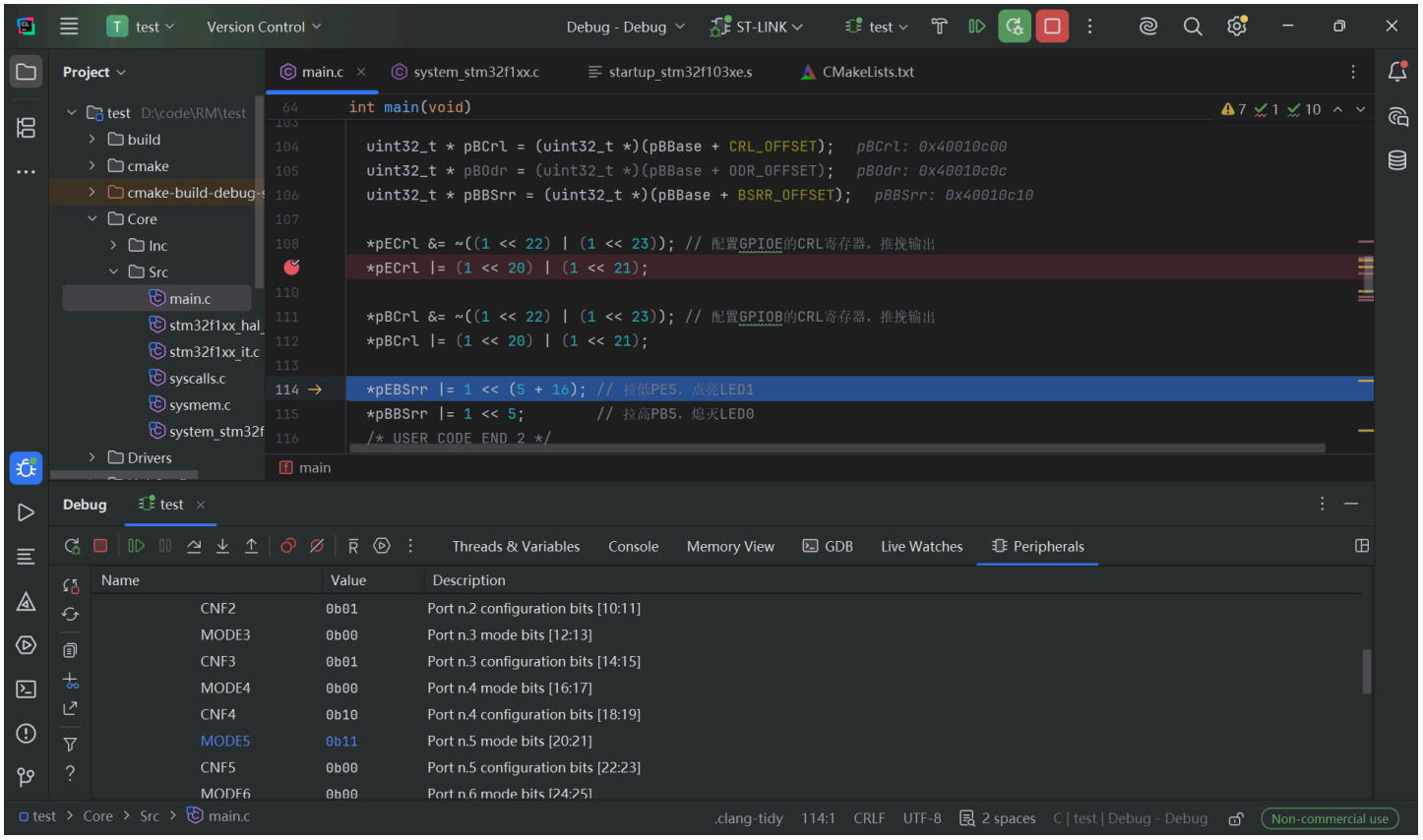


Peripherals：导入svd文件可以查看 / 修改片上外设寄存器

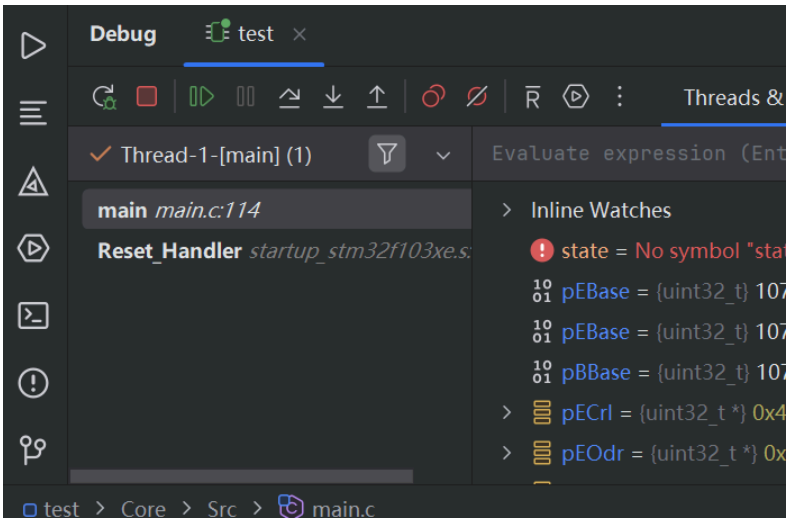
蓝色的为值变化了的寄存器，



将断点打在配置推挽输出后，可以看到GPIOB的Port n.5（PB5）的MODE5寄存器变为0b11(0b表示二进制)，与上面讲的相对应



左侧这个位置显示的是调用堆栈可以把它想象成 “办事记录清单”：代码执行时，A 函数调用了 B 函数，B 函数又调用 C 函数，每进入一个新函数，就会在清单最上面新增一条记录；函数执行结束退出时，就删掉最顶上的记录。清单里保存着：当前正在执行的函数、代码行数、上层是谁调用了它。



> 举个简单例子：`main() → led_init() → gpio_set()`
>
> 堆栈从上到下就是：gpio_set → led_init → main → Reset_Handler
>
> 最上面一行 = 当前正在跑的代码；往下一层层追溯是谁调用的它

附：🖥️实践尝试（需要上交）

以下要求注意代码美观与可读性

建议复杂代码使用注释帮助自己理解

1. 编写一个函数，当参数为奇数的时候点亮LED0，当参数为偶数的时候点亮LED1，并在主函数中调用。

   下面给一个伪代码提供思路

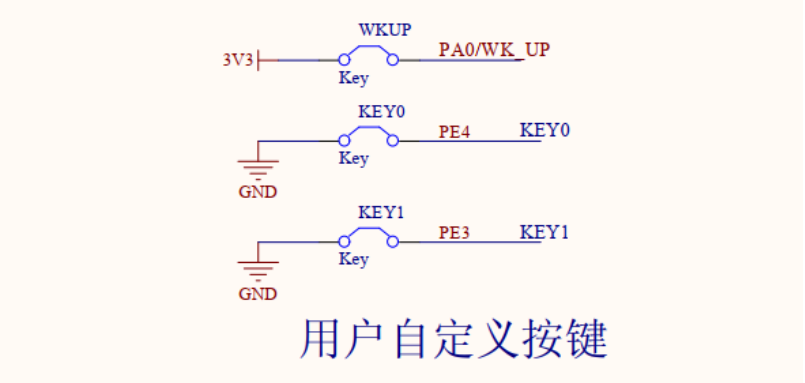
   在主函数中/* USER CODE BEGIN 2 */与/* USER CODE END 2 */之间调用

   伪代码

```
void lightUp(uint8_t num) {
    uint8_t rem = num % 2; // 除2求余数
    if (rem == 1) {
        // 点亮LED0
    } else {
        // 点亮LED1
    }
}
```

- 鼓励直接寄存器操作代替直接使用HAL库，使用GPIO_TypeDef结构体也可以。

2. 实现按下KEY0时LED0亮起，按下KEY1时LED1亮起，以下时按键的硬件原理图



思路：对应引脚配置为GPIO_Input，持续检测电平变化

3. 尝试按一次KEY0闪烁10次LED0，按一次KEY1闪烁5次LED1

附：思考题

1. 本文让LED亮起来为什么要让GPIO保持低电压?
2. 代码是怎么控制GPIO更改电平的?
3. 说明一下启用PD8(或者其他引脚)为GPIO ouput模式和PE3(或者其它引脚)为GPIO 输入上拉模式时，CubeMX的配置流程
4. 如果不使用CubeMX生成的初始化函数，开启GPIO为推挽输出并拉低的简要流程
5. 如果发现LED没有点亮，你会怎么通过Debug排查问题，怎么让程序停止在某一行代码
6.

-- By Scum

QQ: 3014827510